

SmartSafe Master and Slave Units

Complete Design and Code Explanation of Timer2, Timer0, `delay_ms()`, and `delay_us()`

Microprocessors End-Term Project

July 2026

Abstract

This report documents the timing infrastructure used by the SmartSafe ATmega32 Master and Slave units. Timer2 operates as the continuously running application time base. It is configured in Clear Timer on Compare Match mode to produce a compare event approximately every 10 ms, from which software derives 100 ms service ticks and one-second countdowns. Timer0 is reserved as a temporary, polled timing resource for blocking millisecond and microsecond delays, mainly during LCD initialization and command transfers. The report explains every relevant register, the selected prescalers, the calculations leading to `OCR2=77`, `OCR0=124`, and `OCR0=7`, the write-one-to-clear flag behavior, safe interrupt handling, atomic timestamp reads, Timer2 operation in the Master and Slave, and the limitations of software-polled delays on an 8-bit AVR.

Contents

1	Purpose of the Timing Subsystem	3
2	General Operation of an AVR Timer/Counter	3
3	Timer2 as the Periodic System Tick	4
3.1	Why Timer2 is used	4
3.2	Timer2 registers	4
3.3	Timer2 initialization code	4
3.4	Why the timer is stopped first	5
3.5	Counter reset	5
3.6	CTC mode selection	5
3.7	Prescaler selection	5
3.8	Calculation of OCR2	6
3.9	Compare flag and interrupt enable	6
3.10	Why SREG is saved and restored	6
4	Timer2 Interrupt Service Routine in the Master	6
4.1	The 10 ms service level	7
4.2	The 100 ms service level	7
4.3	The one-second service level	7
4.4	Inactivity counter saturation	8
4.5	Countdown counters	8
5	Timer2 Operation in the Slave	8
6	Atomic Reading of the 32-bit Timestamp	9

7	Timer0 as a Blocking Delay Generator	9
7.1	Why Timer0 is used separately	9
7.2	Timer0 registers	10
8	Millisecond Delay Function	10
8.1	Correct implementation	10
8.2	Timer0 CTC mode	10
8.3	Prescaler and OCR0 calculation	11
8.4	Why OCIE0 is disabled	11
8.5	Why the flag is cleared before each interval	11
8.6	Why TCNT0 is reset each time	11
8.7	Accuracy of <code>delay_ms()</code>	12
9	Microsecond Delay Function	12
9.1	Corrected robust implementation	12
9.2	No-prescaler calculation	12
9.3	Why the earlier OCR0=0 form was replaced	13
9.4	Accuracy limitation at very short delays	13
10	Saving and Restoring Timer0 State	13
11	Interaction Between Timer0 Delays and Timer2	14
12	Timer Behavior During Sleep and Power Standby	14
13	Why Long Delays Are Not Implemented with Timer0	14
14	Common Errors and Their Symptoms	15
15	Proteus Verification Procedure	15
15.1	Timer2	15
15.2	<code>delay_ms()</code>	16
15.3	<code>delay_us()</code>	16
16	Design Summary	16
	References	16

1 Purpose of the Timing Subsystem

The SmartSafe program has several activities that must occur at predictable intervals:

- the tamper sensor is serviced approximately every 100 ms;
- the power comparator stability filter is updated approximately every 10 ms;
- the inactivity timer counts toward the 10-second sleep timeout;
- the servo remains open for approximately 5 seconds;
- a security lockout remains active for approximately 30 seconds;
- the Slave keeps LEDs and the alarm active for selected durations;
- LCD commands require short delays measured in microseconds or milliseconds.

A single timing mechanism is not ideal for all these tasks. The implementation therefore separates the responsibilities:

1. **Timer2** is a background, interrupt-driven system tick. It continues to run while ordinary foreground code executes.
2. **Timer0** is temporarily configured for a requested blocking delay. It is polled rather than interrupt-driven, then restored to its earlier state.

Timer1 is not discussed in detail here because it is dedicated to the servo PWM waveform and is documented in the separate servo report.

2 General Operation of an AVR Timer/Counter

An AVR timer contains a binary counter register. A selected clock source advances this counter. The clock can be the CPU clock directly or the CPU clock divided by a prescaler.

For a timer clock derived from the CPU clock,

$$f_{timer} = \frac{F_{CPU}}{N},$$

where N is the prescaler. The duration of one timer count is

$$T_{count} = \frac{N}{F_{CPU}}.$$

In Clear Timer on Compare Match mode, commonly called CTC mode, the counter starts at zero and increments until its value matches the output compare register. A compare flag is then set, and the counter restarts at zero. Since zero is included, the number of timer counts in one CTC period is

$$\text{counts} = OCR + 1.$$

Therefore,

$$T_{CTC} = (OCR + 1) \frac{N}{F_{CPU}},$$

and

$$f_{CTC} = \frac{F_{CPU}}{N(OCR + 1)}.$$

The project assumes

$$F_{CPU} = 8\,000\,000 \text{ Hz.}$$

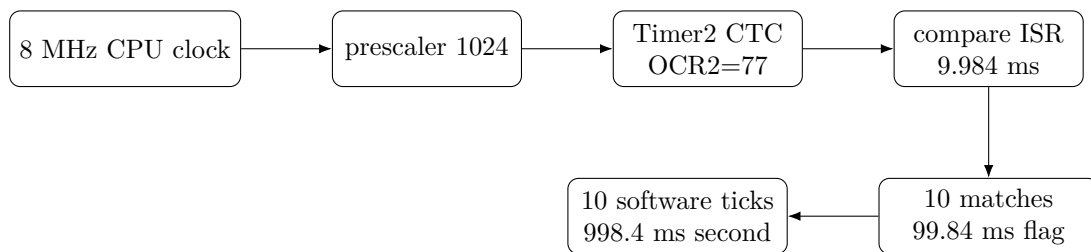
The Proteus clock-frequency property of both ATmega32 devices must also be set to 8 MHz. A mismatch between the simulated clock and the compile-time value changes every timer period, the servo waveform, and the USART baud rate.

3 Timer2 as the Periodic System Tick

3.1 Why Timer2 is used

Timer2 is an 8-bit timer. It is sufficient for generating a short periodic interrupt and leaves Timer1 free for the servo PWM. Timer2 is not used for a long 1-second hardware period directly because an 8-bit counter can represent only 256 counter states. Instead, the hardware produces a period close to 10 ms, and software counts groups of these interrupts.

The selected structure is:



3.2 Timer2 registers

The relevant ATmega32 registers are:

Register	Role
TCCR2	Selects Timer2 waveform mode and clock prescaler.
TCNT2	Holds the current 8-bit counter value.
OCR2	Holds the Timer2 compare value used as TOP in CTC mode.
TIMSK	Contains the Timer2 compare-interrupt enable bit OCIE2.
TIFR	Contains the Timer2 compare flag OCF2; this flag is cleared by writing a logic one.
SREG	Contains the global interrupt-enable bit and is saved while Timer2 is configured atomically.

3.3 Timer2 initialization code

The final initialization pattern is:

```

1 void timer2_init(void)
2 {
3     uint8_t saved_sreg = SREG;
4     cli();
5
6     TCCR2 = 0;
7     TCNT2 = 0;
8     OCR2 = 77U;
9     TIFR = (1U << OCF2);
10
11     sub10ms = 0;
12     sub100ms = 0;
13     timer_tick_100ms = 0;
14

```

```

15     TIMSK |= (1U << OCIE2);
16
17     TCCR2 = (1U << WGM21) |
18             (1U << CS22) |
19             (1U << CS21) |
20             (1U << CS20);
21
22     SREG = saved_sreg;
23 }

```

Listing 1: Timer2 initialization used by the SmartSafe project.

3.4 Why the timer is stopped first

```
1 TCCR2 = 0;
```

With all clock-select bits cleared, Timer2 is stopped. This prevents the counter from advancing while TCNT2, OCR2, flags, and software subcounters are being initialized.

3.5 Counter reset

```
1 TCNT2 = 0;
```

The counter starts from a known value. Without this assignment, a prior Timer2 state could shorten the first interval.

3.6 CTC mode selection

```
1 TCCR2 = (1U << WGM21) | ...;
```

The Timer2 waveform-generation bits are:

WGM21	WGM20	Mode
1	0	CTC; clear Timer2 when TCNT2=OCR2

No Timer2 output pin is required. The project uses only the compare event and its interrupt.

3.7 Prescaler selection

The clock-select bits are:

CS22	CS21	CS20	Timer2 clock
1	1	1	$F_{CPU}/1024$

Thus,

$$f_{T2} = \frac{8\,000\,000}{1024} = 7\,812.5 \text{ Hz.}$$

One Timer2 count takes

$$T_{count} = \frac{1}{7\,812.5} = 128 \text{ } \mu\text{s.}$$

A large prescaler is appropriate because the desired period is close to 10 ms. A smaller prescaler would require more than 256 counts and would not fit in an 8-bit timer.

3.8 Calculation of OCR2

For a target period of approximately 10 ms,

$$OCR2 = \frac{F_{CPU}T}{N} - 1.$$

Substituting $F_{CPU} = 8$ MHz, $T = 0.010$ s, and $N = 1024$:

$$OCR2 = \frac{8\,000\,000 \times 0.010}{1024} - 1 = 77.125.$$

The register must hold an integer. The closest convenient value is 77. The actual period is

$$T_{match} = (77 + 1) \frac{1024}{8\,000\,000} = 9.984 \text{ ms}.$$

The compare frequency is

$$f_{match} = \frac{1}{9.984 \text{ ms}} \approx 100.1603 \text{ Hz}.$$

The use of OCR2=77, rather than 78, follows from the inclusive count sequence:

$$0, 1, 2, \dots, 77.$$

This sequence contains 78 timer counts.

3.9 Compare flag and interrupt enable

```
1 TIFR = (1U << OCF2);
2 TIMSK |= (1U << OCIE2);
```

OCF2 is a write-one-to-clear flag. Assigning a one to this bit removes a stale compare event before interrupts are enabled. This differs from ordinary variables: writing zero does not clear the flag.

Setting OCIE2 allows a Timer2 compare match to request the TIMER2_COMP interrupt. The global interrupt-enable bit in SREG must also be set, normally by `sei()` after system initialization.

3.10 Why SREG is saved and restored

```
1 uint8_t saved_sreg = SREG;
2 cli();
3 ...
4 SREG = saved_sreg;
```

Timer initialization modifies registers shared with interrupt operation. Disabling interrupts makes the configuration sequence atomic. Restoring the saved SREG value is better than always calling `sei()`, because the function preserves the interrupt state chosen by its caller.

4 Timer2 Interrupt Service Routine in the Master

A representative Master ISR is:

```
1 ISR(TIMER2_COMP_vect)
2 {
3     power_monitor_tick_10ms();
4
5     if (++sub10ms < 10U)
6         return;
7 }
```

```

8   sub10ms = 0;
9   timer_tick_100ms = 1;
10
11  if (++sub100ms < 10U)
12      return;
13
14  sub100ms = 0;
15  system_time_s++;
16
17  if (activity_timer < 255U)
18      activity_timer++;
19
20  if (lockout_active && lockout_counter > 0U)
21      lockout_counter--;
22
23  if (servo_open && servo_counter > 0U)
24      servo_counter--;
25  }

```

Listing 2: Master Timer2 compare ISR.

4.1 The 10 ms service level

The power-failure stability filter is serviced on every compare match:

```

1 power_monitor_tick_10ms();

```

Each invocation represents approximately 9.984 ms. A confirmation count of ten ticks therefore represents approximately 99.84 ms.

4.2 The 100 ms service level

```

1 if (++sub10ms < 10U)
2     return;
3
4 sub10ms = 0;
5 timer_tick_100ms = 1;

```

Ten Timer2 periods create

$$10 \times 9.984 = 99.84 \text{ ms.}$$

The ISR sets a flag rather than performing ADC conversion or complex application logic inside the interrupt. The main loop sees the flag, clears it, and calls the tamper-processing function.

This keeps the ISR short and reduces interrupt latency. However, the flag is a single bit-like variable. If the main loop remains blocked for more than one 100 ms interval, multiple periodic events can collapse into one pending flag. That behavior is acceptable for the project because the tamper sensor does not require an exact sample count, but it should be understood as a design tradeoff.

4.3 The one-second service level

Ten 100 ms software periods produce

$$10 \times 99.84 = 998.4 \text{ ms.}$$

At this boundary, the timestamp and application counters are updated.

The difference from an exact second is

$$1000 - 998.4 = 1.6 \text{ ms},$$

which is a relative error of

$$\frac{1.6}{1000} \times 100 = 0.16\%.$$

Therefore the software time base runs approximately 0.16 percent fast. Examples are:

Nominal interval	Actual interval from Timer2	Difference
5 seconds	4.992 seconds	-8 ms
10 seconds	9.984 seconds	-16 ms
30 seconds	29.952 seconds	-48 ms

This error is negligible for the human-scale lockout, sleep, alarm, and servo durations used in the simulation.

4.4 Inactivity counter saturation

```
1 if (activity_timer < 255U)
2     activity_timer++;
```

The variable is eight bits wide. Saturation at 255 prevents wraparound to zero. Without this check, a system left active for 256 seconds could make the inactivity counter appear newly reset.

4.5 Countdown counters

The lockout and servo counters decrement once per software second. The main loop owns the actual state transition. For example, when the servo count reaches zero, foreground code sends the lock command. This separation avoids calling peripheral drivers from inside Timer2's ISR.

5 Timer2 Operation in the Slave

The Slave uses the same hardware values:

```
1 OCR2 = 77U;
2 TCCR2 = (1U << WGM21) |
3         (1U << CS22) |
4         (1U << CS21) |
5         (1U << CS20);
```

The Slave ISR has fewer responsibilities. It converts 100 Timer2 compare events into one software second and decrements the red-alarm and green-success countdowns:

```
1 ISR(TIMER2_COMP_vect)
2 {
3     if (++sub10ms < 10U)
4         return;
5
6     sub10ms = 0;
7
8     if (++sub100ms < 10U)
9         return;
10
11    sub100ms = 0;
12
13    if (alarm_counter > 0U)
```



```

14     alarm_counter--;
15
16     if (green_counter > 0U)
17         green_counter--;
18 }

```

Listing 3: Simplified Slave Timer2 ISR.

The main loop observes the counters and turns outputs off when they reach zero. This makes Timer2 a shared scheduler without placing LCD or buzzer operations inside the interrupt.

6 Atomic Reading of the 32-bit Timestamp

The ATmega32 is an 8-bit CPU, while `system_time_s` is a 32-bit value. Reading the variable requires several machine accesses. Timer2 could interrupt between these accesses and increment the value, producing a mixed result.

The safe accessor is:

```

1 uint32_t timer_get_seconds(void)
2 {
3     uint32_t value;
4     uint8_t saved_sreg = SREG;
5
6     cli();
7     value = system_time_s;
8     SREG = saved_sreg;
9
10    return value;
11 }

```

Disabling interrupts for only the short copy guarantees that all four bytes belong to the same timestamp. The original interrupt state is then restored.

7 Timer0 as a Blocking Delay Generator

7.1 Why Timer0 is used separately

Timer2 must remain configured as the system tick. Reconfiguring Timer2 for an LCD delay would disturb the lockout, tamper, power-filter, and servo timing. Timer0 is therefore used as a temporary timing resource.

The delay functions do not enable a Timer0 interrupt. They poll the compare flag directly:



The CPU remains busy inside the delay loop, so these functions are blocking. Global interrupts remain enabled, which means Timer2 can normally continue running while the foreground waits.

7.2 Timer0 registers

Register	Role
TCCR0	Selects Timer0 CTC mode and prescaler.
TCNT0	Holds the current Timer0 count.
OCR0	Holds the compare value that defines one delay unit.
TIMSK	Contains OCIE0; the delay functions disable this interrupt because they poll instead.
TIFR	Contains OCF0; writing one clears the compare flag.

8 Millisecond Delay Function

8.1 Correct implementation

```

1 void delay_ms(uint16_t ms)
2 {
3     uint8_t saved_tccr0 = TCCR0;
4     uint8_t saved_timsk = TIMSK;
5     uint8_t saved_ocr0  = OCR0;
6     uint8_t saved_tcnt0 = TCNT0;
7
8     TCCR0 = 0;
9     TIMSK &= ~(1U << OCIE0);
10
11     OCR0  = 124U;
12     TCNT0 = 0;
13     TIFR  = (1U << OCF0);
14
15     TCCR0 = (1U << WGM01) |
16             (1U << CS01)  |
17             (1U << CS00);
18
19     while (ms-- > 0U) {
20         TCNT0 = 0;
21         TIFR  = (1U << OCF0);
22
23         while (!(TIFR & (1U << OCF0))) {
24             /* blocking wait */
25         }
26     }
27
28     TCCR0 = 0;
29     TIFR  = (1U << OCF0);
30     OCR0  = saved_ocr0;
31     TCNT0 = saved_tcnt0;
32     TIMSK = saved_timsk;
33     TCCR0 = saved_tccr0;
34 }

```

Listing 4: Timer0-based blocking millisecond delay.

8.2 Timer0 CTC mode

For Timer0,

WGM01	WGM00	Mode
1	0	CTC using OCR0

The project does not use the OC0 output pin. It only waits for the internal compare flag.

8.3 Prescaler and OCR0 calculation

The millisecond function selects a prescaler of 64:

CS02	CS01	CS00	Timer0 clock
0	1	1	$F_{CPU}/64$

Thus,

$$f_{T0} = \frac{8\,000\,000}{64} = 125\,000 \text{ Hz.}$$

One count takes

$$T_{count} = \frac{1}{125\,000} = 8 \mu s.$$

The number of counts required for 1 ms is

$$\frac{1 \text{ ms}}{8 \mu s} = 125.$$

Since the timer counts from zero through the compare value,

$$OCR0 = 125 - 1 = 124.$$

The ideal hardware compare interval is therefore

$$(124 + 1) \times 8 \mu s = 1000 \mu s = 1 \text{ ms.}$$

8.4 Why OCIE0 is disabled

```
1 TIMSK &= ~(1U << OCIE0);
```

The function waits for OCF0 itself. Enabling the Timer0 compare interrupt would be unnecessary and could invoke an unrelated ISR. Only the Timer0 interrupt-enable bit is changed; the saved TIMSK value is restored afterward.

8.5 Why the flag is cleared before each interval

```
1 TIFR = (1U << OCF0);
```

A previous compare may have left OCF0=1. If the stale flag were not cleared, the polling loop could terminate immediately and produce a zero-length interval. The ATmega32 clears this flag by writing a logic one.

8.6 Why TCNT0 is reset each time

```
1 TCNT0 = 0;
```

Resetting the counter establishes a complete 125-count interval for each requested millisecond. It also makes the function independent of the count value that existed before the current loop iteration.

8.7 Accuracy of delay_ms()

The timer portion of each iteration is exactly 1 ms for an ideal 8 MHz CPU clock. The complete function is slightly longer because software executes loop control, register writes, and the polling branch. Therefore,

$$T_{actual} \gtrsim ms \times 1 \text{ ms}.$$

The overhead is small compared with a millisecond and is acceptable for LCD power-up, clear-display, and human-interface pauses. It should not be treated as a laboratory-grade time reference.

9 Microsecond Delay Function

9.1 Corrected robust implementation

The reliable final implementation uses the full 8 MHz Timer0 clock and eight timer counts for each nominal microsecond:

```

1 void delay_us(uint16_t us)
2 {
3     uint8_t saved_tccr0 = TCCR0;
4     uint8_t saved_timsk = TIMSK;
5     uint8_t saved_ocr0 = OCR0;
6     uint8_t saved_tcnt0 = TCNT0;
7
8     TCCR0 = 0;
9     TIMSK &= ~(1U << OCIE0);
10
11     OCR0 = 7U;
12     TCNT0 = 0;
13     TIFR = (1U << OCF0);
14
15     TCCR0 = (1U << WGM01) |
16             (1U << CS00);
17
18     while (us-- > 0U) {
19         TCNT0 = 0;
20         TIFR = (1U << OCF0);
21
22         while (!(TIFR & (1U << OCF0))) {
23             /* blocking wait */
24         }
25     }
26
27     TCCR0 = 0;
28     TIFR = (1U << OCF0);
29     OCR0 = saved_ocr0;
30     TCNT0 = saved_tcnt0;
31     TIMSK = saved_timsk;
32     TCCR0 = saved_tccr0;
33 }
```

Listing 5: Corrected Timer0-based microsecond delay.

9.2 No-prescaler calculation

With CS00=1 and the other clock-select bits zero,

$$f_{T0} = F_{CPU} = 8\,000\,000 \text{ Hz}.$$

One timer count is

$$T_{count} = \frac{1}{8\,000\,000} = 0.125\ \mu\text{s}.$$

Eight counts produce

$$8 \times 0.125\ \mu\text{s} = 1\ \mu\text{s}.$$

Since CTC counts from zero through OCR0,

$$OCR0 = 8 - 1 = 7.$$

Thus, the hardware comparison interval is nominally 1 microsecond.

9.3 Why the earlier OCR0=0 form was replaced

An earlier version selected a prescaler of 8 and used OCR0=0. Mathematically, one divided timer count equals 1 microsecond at 8 MHz. However, a zero compare value creates an extremely short compare sequence and behaved unreliably in the Proteus simulation used for this project. The LCD initialization could stall while waiting for a flag transition.

The corrected form uses eight physical counter states at the undivided clock:

```
1 OCR0 = 7U;
2 TCCR0 = (1U << WGM01) | (1U << CS00);
```

This represents the same ideal microsecond duration but provides a clearer and more robust simulated timer interval.

9.4 Accuracy limitation at very short delays

For `delay_us(1)`, software overhead is comparable to or greater than the requested timer interval. Register writes, loop branches, and function call/return instructions all consume CPU cycles. Therefore the function guarantees a safe minimum delay rather than an exact one-microsecond wall-clock interval:

$$T_{actual} > 1\ \mu\text{s} \quad \text{for very small arguments.}$$

This is appropriate for the HD44780-compatible LCD, where delays must be at least a specified minimum. It is not appropriate for generating a high-precision communication waveform. Hardware PWM, SPI, USART, or output compare should be used for exact high-speed signaling.

10 Saving and Restoring Timer0 State

Both delay functions save:

```
1 TCCR0, TIMSK, OCR0, TCNT0
```

This allows Timer0 to be borrowed without permanently destroying a configuration established elsewhere.

The restoration order is deliberate:

1. set TCCR0=0 to stop Timer0;
2. clear the final compare flag;
3. restore OCR0 and TCNT0;
4. restore TIMSK;

5. restore TCCR0 last, so the previous clock starts only after all associated state is ready.

The functions do not save TIFR, because interrupt flags are event indicators rather than ordinary configuration state. The delay clears its own compare event before returning.

11 Interaction Between Timer0 Delays and Timer2

Timer0 polling blocks the foreground code, but it does not call `cli()`. Therefore Timer2 interrupts remain enabled and continue to update system timing during most delay calls.

For example, during a 1.5-second LCD splash delay:

- Timer0 repeatedly generates the millisecond wait intervals;
- Timer2 still interrupts approximately every 9.984 ms;
- the application timestamp and inactivity counter continue to advance;
- the foreground cannot scan the keypad or service a one-bit 100 ms flag until the delay returns.

This is why blocking delays should be kept short during normal operation. Long human-scale behaviors, such as the 5-second servo interval and 30-second lockout, are implemented with Timer2 counters rather than `delay_ms()`.

12 Timer Behavior During Sleep and Power Standby

Timer operation depends on the selected AVR sleep mode and on whether the firmware explicitly stops a timer.

In ordinary inactivity Power-down sleep, synchronous timers driven from the CPU clock stop. Timer2 therefore does not count the sleep duration in the current design. The external interrupt on INT0 wakes the Master, after which normal timing resumes.

During power-failure standby, the firmware preserves the analog comparator as the restoration wake source and may disable ordinary runtime peripherals, including Timer2. The comparator transition wakes the CPU, the power module accepts restoration, and the runtime initialization restores Timer2.

The blocking Timer0 functions must not be called while the CPU is already sleeping. They require a running CPU clock because the foreground actively polls `OCF0`.

13 Why Long Delays Are Not Implemented with Timer0

A call such as

```
1 delay_ms(30000U);
```

would block the main loop for 30 seconds. During that interval, the system could not process keypad input, USART commands, state transitions, or ordinary foreground events. Although Timer2 interrupts might continue, the architecture would become unresponsive.

The project instead uses nonblocking application counters:

```
1 lockout_counter = 30U;  
2 servo_counter  = 5U;
```

Timer2 decrements them in the background while the main loop remains available for other work.

The design rule is:

Timing requirement	Mechanism
LCD setup and bus timing	Timer0 blocking delay
Short initialization pause	Timer0 blocking delay
Periodic 10 ms and 100 ms service	Timer2 interrupt
5-second servo interval	Timer2 software countdown
10-second inactivity interval	Timer2 software counter
30-second lockout or breach alarm	Timer2 software countdown
Precise servo pulse train	Timer1 hardware PWM

14 Common Errors and Their Symptoms

Error	Consequence	Correction
Proteus MCU clock left at 1 MHz	All timer intervals become approximately eight times too long; USART and servo also fail	Set both ATmega32 clock properties to 8 MHz
Forgetting the +1 in CTC calculations	Compare periods are off by one timer count	Use $T = (OCR + 1)N/F_{CPU}$
Clearing OCF0 or OCF2 by writing zero	Stale flag remains set	Clear by writing a logic one
Using OCR0=0 for the microsecond delay in Proteus	Possible simulation stall or unreliable LCD timing	Use no prescaler and OCR0=7
Not resetting TCNT0	First delay interval can be shortened	Set TCNT0=0 for each unit
Reconfiguring Timer2 for a delay	System tick and countdowns are corrupted	Reserve Timer2 for periodic scheduling and use Timer0 for delays
Using a long blocking delay for lockout	Keypad, USART, and main-loop processing freeze	Use Timer2 countdown variables
Reading <code>system_time_s</code> without atomic protection	A mixed 32-bit timestamp may be obtained	Use <code>timer_get_seconds()</code>
Leaving OCIE0 enabled while polling	An unwanted Timer0 ISR may run	Temporarily clear OCIE0 and restore TIMSK later
Restoring Timer0 clock before its compare/count values	Previous timer configuration can run briefly with wrong state	Restore TCCR0 last

15 Proteus Verification Procedure

15.1 Timer2

A practical verification method is to toggle an unused debug pin in the Timer2 ISR and observe it with the Proteus oscilloscope or logic analyzer. Toggling on every compare produces a square wave whose half-period is approximately 9.984 ms and full period is approximately 19.968 ms. Setting a pin high for one ISR and low for the next can also reveal the interrupt interval.

The 100 ms flag can be tested by toggling another pin only when `sub10ms` reaches ten. The observed event spacing should be approximately 99.84 ms.

15.2 `delay_ms()`

Set a debug output high, call `delay_ms(100)`, then set it low. The high pulse should be slightly longer than 100 ms because of software overhead.

15.3 `delay_us()`

For a visible logic-analyzer test, use a larger argument such as `delay_us(100)`. Very short calls are dominated by software overhead and simulation resolution. The function is intended to satisfy minimum LCD timing, not to serve as a calibrated pulse generator.

16 Design Summary

Timer2 is the SmartSafe scheduler. With an 8 MHz CPU clock, prescaler 1024, CTC mode, and `OCR2=77`, it produces a 9.984 ms compare event. Software groups ten events into a 99.84 ms service flag and ten of those groups into a 998.4 ms application second. This supplies the power confirmation filter, tamper service flag, timestamps, sleep timeout, servo interval, lockout, and Slave indicator countdowns.

Timer0 is a temporary blocking delay resource. For milliseconds, prescaler 64 and `OCR0=124` create an ideal 1 ms compare interval. For microseconds, the corrected implementation uses no prescaler and `OCR0=7`, producing an ideal one-microsecond hardware interval from eight 0.125-microsecond counts. Both functions poll `OCF0`, preserve the previous Timer0 configuration, and restore the timer safely after use.

The central design principle is separation of timing roles: Timer0 handles short blocking hardware delays, Timer2 handles responsive application scheduling, and Timer1 independently generates the servo PWM waveform.

References

1. Microchip Technology Inc., *ATmega32A Data Sheet*, DS40002072A. [Official Microchip PDF](#).
2. Microchip Technology Inc., *AVR Instruction Set Manual*, DS40002198A. [Official Microchip PDF](#).
3. SmartSafe project source files: `timer.c`, `timer.h`, `config.h`, and the Master/Slave main-loop modules.